



Exercício 1: Encapsulamento - O Guardião dos Atributos

Contexto

Você está desenvolvendo a lógica de um personagem de RPG. Os atributos vitais (vida e nível) não podem ser acessados diretamente para evitar estados inválidos (como vida negativa ou nível alterado sem ganhar experiência).

Código Cliente (Não modificar)

Copie este código para sua IDE. Sua classe `PersonagemRPG` deve satisfazer estas chamadas.

```
int main() {
    std::cout << "--- Início do Jogo ---\n";
    PersonagemRPG heroi("Aragorn");
    heroi.exibirStatus();

    // Teste de Dano
    std::cout << "\n--- Batalha contra Orc ---\n";
    heroi.receberDano(30);
    heroi.exibirStatus();

    // TENTATIVA DE TRAPAÇA (Deve dar erro se descomentado)
    // heroi.vidaAtual = 1000;
    // heroi.nivel = 50;

    // Teste de Cura
    std::cout << "\n--- Bebendo Poção ---\n";
    heroi.curar(200); // Nao deve passar da vida maxima!
    heroi.exibirStatus();

    // Teste de Lógica de Nível
    std::cout << "\n--- Ganho de Experiência ---\n";
    heroi.ganharExperiencia(80);
    heroi.ganharExperiencia(30); // Total 110 XP -> Level Up + Sobra 10
    XP

    std::cout << "\n--- Status Final ---\n";
    heroi.exibirStatus();

    return 0;
}
```

Requisitos da Classe

- **Atributos Privados:** Nome, Nível (inicia em 1), Vida Atual, Vida Máxima (Nível * 50), Experiência (inicia em 0).
- **Métodos Públicos:**
 - `receberDano(int)`: Reduz a vida. Mínimo zero.
 - `curar(int)`: Aumenta a vida. Não pode ultrapassar Vida Máxima.
 - `ganharExperiencia(int)`: Soma XP. A cada 100 XP, sobe de nível, recalcula Vida Máxima e cura totalmente.

Exercício 2: Herança - O Estoque Inteligente

Contexto

Implemente um sistema de inventário para uma loja que vende Livros e Eletrônicos. Use herança para evitar repetição de código (como nome e preço) que é comum a todos os produtos.

Código Cliente (Não modificar)

```
int main() {
    std::cout << "--- Cadastro de Produtos ---\n";

    // Produto Generico (Teste da base)
    Produto p1("Suporte para Notebook", 50.00);
    p1.exibirDetalhes();
    std::cout << "\n";

    // Livro (Herda de Produto)
    Livro l1("O Senhor dos Aneis", 120.00, "J.R.R. Tolkien");
    l1.exibirDetalhes();
    std::cout << "\n";

    // Eletronico (Herda de Produto)
    Eletronico e1("Fone Bluetooth", 200.00, "JBL");
    e1.exibirDetalhes();

    return 0;
}
```

Requisitos das Classes

1. Classe Base Produto:

- Atributos `protected`: nome, preço.
- Método: `exibirDetalhes()` (Mostra nome e preço).

2. Classe Livro (Herda de Produto):

- Atributo específico: autor.
- Construtor: Deve chamar o construtor de `Produto`.
- Método: `exibirDetalhes()` deve mostrar dados do pai + o autor.

3. Classe Eletronico (Herda de Produto):

- Atributo específico: marca.
- Método: `exibirDetalhes()` deve mostrar dados do pai + a marca.

Saída Esperada:

Produto: Suporte para Notebook | Preço: R\$ 50.00

Produto: O Senhor dos Aneis | Preço: R\$ 120.00

Autor: J.R.R. Tolkien

Produto: Fone Bluetooth | Preço: R\$ 200.00

Marca: JBL

Exercício 3: Polimorfismo - O Notificador Universal

Contexto

Crie um sistema de alertas onde uma única mensagem de erro possa ser enviada por diferentes canais (Email, SMS, Push) usando Polimorfismo. O sistema deve ser extensível sem uso de `if/else` baseados em tipo.

Código Cliente (Não modificar)

```
int main() {
    std::cout << "--- Sistema de Alerta de Servidores ---\n";

    // Vetor de ponteiros para a classe BASE
    std::vector<Notificacao*> canais;

    canais.push_back(new Email("admin@sys.com", "FALHA CRITICA"));
    canais.push_back(new SMS("11-99999-8888"));
    canais.push_back(new PushNotification("Device-X", "MonitorApp"));

    std::cout << ">>> Ocorreu um erro no Servidor DB-01! Enviando\n" << "alertas..." << std::endl;

    std::string erro = "Servidor offline.";

    // Loop Polimorfico
    for (Notificacao* canal : canais) {
        canal->enviar(erro); // Cada objeto reage de uma forma
        std::cout << "\n";
    }

    // Limpeza
    for (auto c : canais) delete c;
    canais.clear();

    return 0;
}
```

Requisitos das Classes

1. Classe Abstrata Notificacao:

- Método puramente virtual: `virtual void enviar(string msg) = 0;`
- Destrutor virtual obrigatório.

2. Classes Derivadas (Email, SMS, PushNotification):

- Cada uma deve implementar o método `enviar` formatando a saída de maneira única (ex: SMS exibe número, Email exibe assunto).

Conceito Chave

Lembre-se: Para o polimorfismo funcionar em C++, você deve usar a palavra-chave `virtual` na classe base e trabalhar com ponteiros ou referências na `main`.

Saída Esperada:

```
--- Sistema de Alerta de Servidores ---
```

```
>>> Ocorreu um erro no Servidor DB-01! Enviando alertas...
```

```
[EMAIL] Para: admin@empresa.com
```

```
Assunto: FALHA CRITICA
```

```
Corpo: O servidor parou de responder.
```

```
[SMS] Enviando para 11-99999-8888: O servidor parou de responder.
```

```
[PUSH] App: MonitorApp | Device: Device-X
```

```
Msg: O servidor parou de responder.
```